

Rust en Producción

Lecciones del podcast Software Engineering Radio

Integrantes:

Richard Robin Roque del Rio - UO302698

Fernando Remis Figueroa - UO302109

Sergio Argüelles Huerta - UO299741

Cómo desarrollar, desplegar y mantener aplicaciones Rust reales basadas en la experiencia de Luca Palmieri, autor de "Zero to Production in Rust".



Contexto y estructura de la presentación

Esta presentación está basada en un episodio del podcast **Software Engineering Radio** donde entrevistan a **Luca Palmieri**, autor del libro "Zero to Production in Rust". El episodio explora cómo desarrollar, desplegar y operar aplicaciones Rust en entornos productivos.



Desarrollo

Aplicaciones Rust, Inner Development Loop, estructura de proyectos



Deployment

Compilación, Docker, multi-stage builds, arquitectura



Rendimiento

Métricas, telemetría, manejo de errores, optimización



Qué significa "Rust en producción"

El código en producción debe cumplir requisitos fundamentales para operar software real:

Estabilidad

El software debe funcionar consistentemente sin crashes inesperados en entornos críticos.

Monitorización

Capacidad de observar métricas, logs y estado del sistema en tiempo real.

Recuperación ante fallos

El sistema debe recuperarse automáticamente de errores sin intervención manual.

Escalabilidad

Funcionamiento eficiente a gran escala con múltiples usuarios concurrentes.

Compilación en Rust



Rust compila el código a **binarios nativos** altamente optimizados. El proceso de compilación tiene dos configuraciones principales:

Debug

- Compilación rápida para desarrollo
- Sin optimizaciones del compilador
- Mayor información de depuración
- Peor rendimiento en ejecución

Release

- Compilación más lenta
- Optimizaciones completas
- Binarios pequeños y eficientes
- Mejor rendimiento final



En producción siempre se utiliza la configuración **release** para obtener el máximo rendimiento.

Optimización del compilador

Dos técnicas avanzadas para mejorar el rendimiento de aplicaciones Rust en producción:



1

Link Time Optimization (LTO)

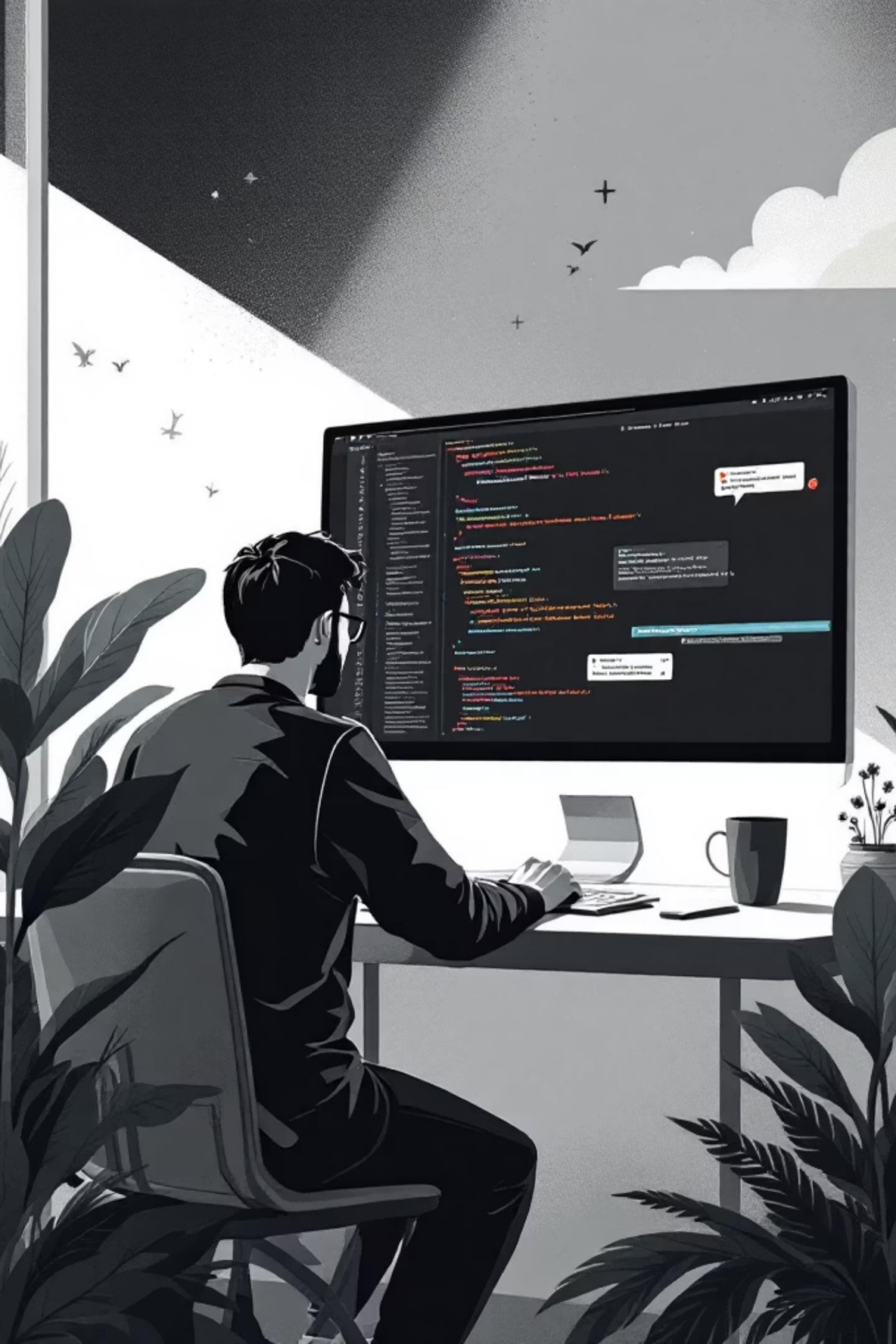
Optimiza el código entre diferentes librerías durante el enlazado. Reduce la sobrecarga de llamadas entre crates y mejora el rendimiento final del binario ejecutable.



2

Profile Guided Optimization (PGO)

Utiliza datos reales de ejecución para optimizar las partes del código más utilizadas. Se ejecuta el programa con perfiles de uso típicos y el compilador ajusta las optimizaciones según patrones reales.



Desarrollo y Despliegue en Rust

Optimizando el ciclo de desarrollo, estructurando proyectos escalables y desplegando aplicaciones eficientes en producción.

El Inner Development Loop

El ciclo que repite un desarrollador cientos de veces al día: escribir código, compilar, probar y corregir errores.

1

Escribir

Implementar nueva funcionalidad o modificar código existente

2

Compilar

El compilador verifica la corrección del código

3

Testear

Ejecutar pruebas para verificar el comportamiento

4

Corregir

Iterar y mejorar según los resultados obtenidos

📌 **El desafío de Rust:** Su compilador es extremadamente riguroso, lo que garantiza seguridad pero puede ralentizar el ciclo. Por eso es crucial optimizar este bucle con herramientas como cargo check y compilación incremental.

Proceso de Despliegue

01

Compilación

Generar el binario ejecutable con
cargo build --release

02

Empaquetado

Crear contenedores Docker o
artefactos para despliegue

03

Despliegue

Implementar en servidores o plataformas cloud



Estructura de Proyectos: Crates y Workspaces

El concepto de Crate

En Rust, la unidad básica es el **crate**: una librería o módulo independiente que puede compilarse como parte de un proyecto.

Los proyectos pequeños usan un solo crate, pero los sistemas grandes se benefician de dividir el código en múltiples crates dentro de un **workspace**.



Organización

Código modular, limpio y fácil de mantener

Reutilización

Componentes compartibles entre diferentes microservicios

Velocidad

Compilación incremental: solo recompila lo modificado

Optimización con Docker Multi-stage

Una técnica que combina eficiencia en compilación con imágenes mínimas para producción.

Etapa 1: Compilación

Usa una imagen completa con todas las herramientas de Rust para compilar el código

Etapa 2: Extracción

Copia solo el binario final a una imagen mínima (Alpine o Distroless)

10-20MB

Tamaño reducido

Imágenes Docker extremadamente ligeras

0

Sin dependencias

Binarios autocontenidos sin instalaciones externas

100%

Seguridad

Sin herramientas sobrantes que puedan explotarse

Conclusión

Optimización del ciclo

Herramientas como `cargo check` y compilación incremental mantienen el feedback rápido

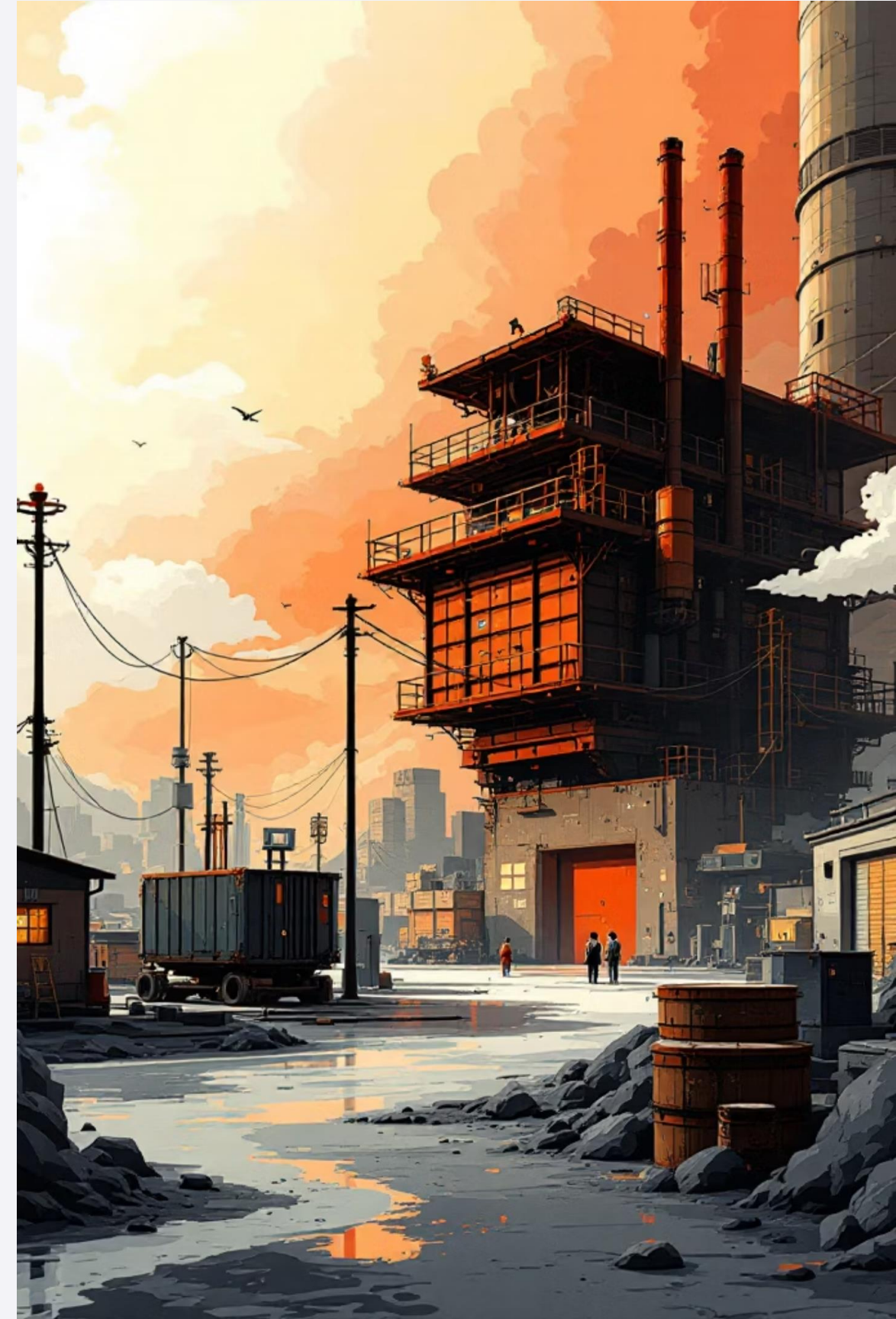
Modularización efectiva

Crates y workspaces permiten proyectos escalables y reutilizables

Despliegue eficiente

Binarios autocontenidos y contenedores ligeros garantizan velocidad y seguridad

Rust combina **seguridad**, **rendimiento** y **portabilidad** para infraestructura crítica en producción.





Rendimiento, telemetría y manejo de errores

Rendimiento



En aplicaciones backend, tres métricas definen el rendimiento real del sistema:

Peticiones/s

Rust procesa cargas altas con un uso de CPU muy eficiente.

Memoria

Consumo bajo y predecible.

Latencia de las respuestas

Estable y sin pausas gracias a la ausencia de garbage collector.

La diferencia clave

**Sin garbage collector, sin pausas
inesperadas.**

En sistemas de producción, la predictibilidad de la latencia es tan importante como la velocidad máxima. Rust gestiona la memoria en tiempo de compilación, eliminando por completo las pausas de recolección de basura que afectan a lenguajes como Java.

Telemetría y observabilidad

¿Qué permite **tracing**?

→ Logs estructurados

Registro con campos organizados, no solo texto

→ Registro de eventos

Trazabilidad completa del ciclo de vida de cada petición.

→ Medir la duración de las operaciones



La información recopilada se envía a sistemas de monitorización para analizar el comportamiento y detectar anomalías a tiempo.

Manejo de errores

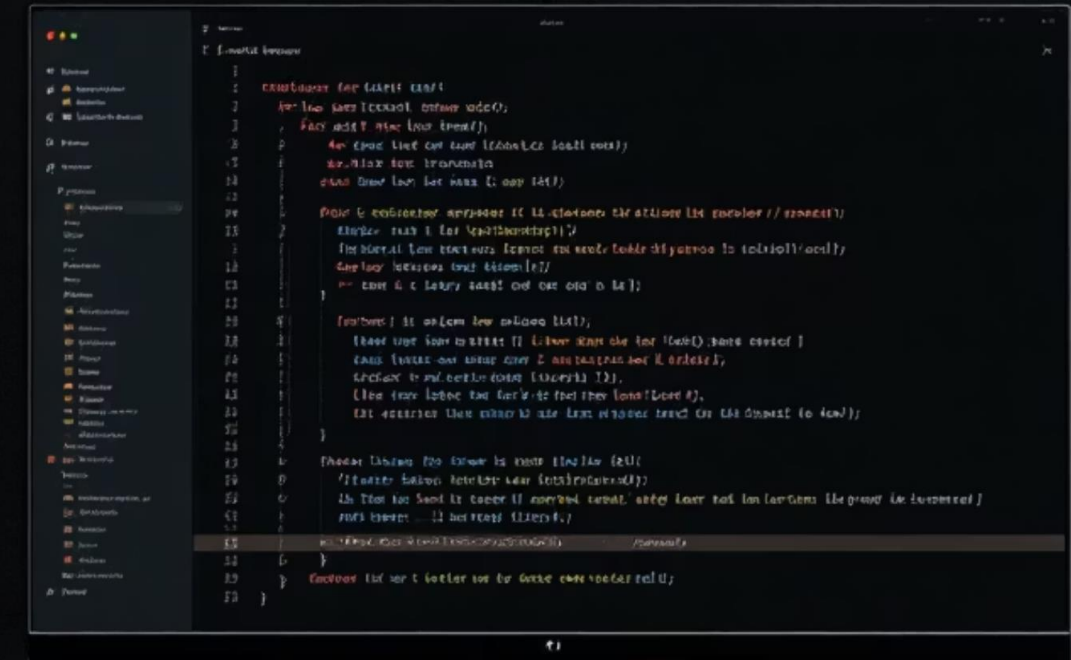
Rust abandona las excepciones implícitas. Todos los fallos se representan con el tipo `Result`, que obliga a tomar una decisión explícita ante cada posible error.

Otros lenguajes

Excepciones que se propagan de forma implícita y pueden pasar desapercibidas.

Rust con `Result`

El compilador exige gestionar cada fallo. Los errores silenciosos son imposibles.

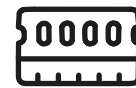


Conclusión

Rust se consolida como una opción de primera clase para sistemas backend en producción, combinando cuatro pilares esenciales:



Alto rendimiento



Bajo consumo de memoria



Errores explícitos



Buenas herramientas de observabilidad

Rust no es solo una tendencia — es una apuesta técnica sólida para construir infraestructura eficiente, estable y predecible.